

Object Initialization (CodeGrade)

 <https://github.com/learn-co-curriculum/python-p3-init-self>  <https://github.com/learn-co-curriculum/python-p3-init-self/issues/new>

Learning Goals

- Learn about the `__init__` method.
 - Learn about the `self` keyword.
 - Initialize unique objects from the same class.
-

Key Vocab

- **Class:** a bundle of data and functionality. Can be copied and modified to accomplish a wide variety of programming tasks.
 - **Initialize:** create a working copy of a class using its `__init__()` method.
 - **Instance:** one specific working copy of a class. It is created when a class's `__init__()` method is called.
 - **Object:** the more common name for an instance. The two can usually be used interchangeably.
 - **Function:** a series of steps that create, transform, and move data.
 - **Method:** a function that is defined inside of a class.
 - **self:** a keyword reference to the current instance of the class that is used to access attributes of the instance.
 - **Attribute:** variables that belong to an object.
-

Introduction

We've already seen many new instances of different classes being created:

```
class Dog:
    pass

fido = Dog()
# <__main__.Dog object at 0x104fc0550>

snoopy = Dog()
# <__main__.Dog object at 0x104fc0562>
```

We know from their IDs that `fido` and `snoopy` live in different locations in memory, but their *behavior* is identical. We don't provide them any unique methods or attributes in the code above.

We can modify their attributes using dot notation as follows:

```
fido.breed = "Dalmatian"
snoopy.breed = "Beagle"

fido.breed
# Dalmatian
snoopy.breed
# Beagle
```

...but dogs are *born* with breeds, not assigned breeds later on. To provide objects with unique attributes upon instantiation, we use the `__init__` method.

`__init__`

Every time we initialize a new object, the Python interpreter calls a **magic method** called `__init__`. **Magic methods** (also called **dunder methods** due to their **double underscores**) are special methods that are not meant to be called by developers but are invoked by different cues and operators that act on their objects.

The `__init__` method belongs to every class and is invoked when you initialize a class. Though you cannot see it in our `Dog` class above, there is code that is executed behind the scenes.

Though it always executes when we initialize an object, it does not do anything by default. Instead, we overwrite `__init__` in most classes to change an object's behavior:

```
class Dog:
    # Remember that all methods use "self." We'll elaborate on this in a moment.
    def __init__(self):
        print("A dog is born!")

    def bark(self):
        print("Woof!")

fido = Dog()
# A dog is born!
```

`__init__` can also be modified to take more arguments. This allows us to customize the behavior of unique objects:

```
class Dog:
    def __init__(self, name):
        print(f"{name} is born!")

    def bark(self):
        print("Woof!")

fido = Dog("Fido")
# Fido is born!

snoopy = Dog("Snoopy")
# Snoopy is born!
```

► *When is the `__init__` magic method run?*

self

Modifying `__init__` has allowed us to provide `fido` and `snoopy` with different behaviors, but now that they've both been initialized, they're more or less the same dog. In order to make lasting changes to an object, we need to modify `self`. Before we do, open up the Python shell and let's take a look at `self`:

```
class Dog:
    def __init__(self, name):
        print(f"{name} is born!")

    def bark(self):
        print("Woof!")

    def showing_self(self):
        return self

fido = Dog("Fido")
# Fido is born!
fido.showing_self()
# <__main__.Dog object at 0x104f8bfa0>
```

You might recognize this output from earlier- `self` is the same type of object as `fido` ! As a matter of fact, it *is* `fido` !

```
fido is fido.showing_self()
# True
```

How does this work? Inside the `showing_self()` method, we use the `self` keyword.

The `self` keyword **refers to the instance**, or object, that the `showing_self()` method is being called on.

So, when we call `showing_self()` on `fido`, the method will return the `Dog` instance that is `fido`.

If you're familiar with Object Oriented JavaScript from previous experience, the closest equivalent to `self` in Python is `this` in JavaScript. Again, no worries if you haven't seen OOJS before. The `this` keyword behaves similarly, where `this` refers to **the object to the left of the dot** when a method is called on an object:

```
class Dog {  
  showingThis() {  
    return this;  
  }  
}  
  
const fido = new Dog();  
fido.showingThis();  
// => Dog {}
```

`__init__` and `self` Together

Now that we know that `__init__` contains code that runs whenever we initialize a new object and that the `self` that is passed to each method is a reference to the initialized object, let's make some unique dogs:

```
class Dog:  
    def __init__(self, name):  
        self.name = name  
  
    def bark(self):  
        print("Woof!")  
  
fido = Dog("Fido")  
fido.name  
# Fido
```

```
snoopy = Dog("Snoopy")
snoopy.name
# Snoopy
```

Operating on `self` in an Instance Method

Let's say that Fido here is getting adopted. Fido's new owner is Sophie. We can set Fido's `owner` attribute equal to the string of `"Sophie"`. The name of his new owner:

```
class Dog:
    def __init__(self, name):
        self.name = name

fido.owner = "Sophie"

fido.owner
# "Sophie"
```

Great, Fido now knows the name of his owner. Let's think about the situation in which `fido` gets a new owner. This would occur at the moment in which `fido` is adopted.

To represent this with code, we could write an `adopt()` function like this:

```
def adopt(dog, owner_name):
    dog.owner = owner_name
```

Here we have a method that takes in two arguments, an instance of the `Dog` class and an owner's name. We could call our method like this:

```
adopt(fido, "Sophie")
```

```
# now we can ask Fido who his owner is:
```

```
fido.owner  
# => "Sophie"
```

However, the beauty of object-oriented programming is that we can encapsulate, or wrap up, attributes and behaviors into one object. Instead of writing a function that is not associated to any particular object and that takes in certain objects as arguments, we can simply teach our **Dog** instances how to get adopted.

Let's refactor our code above into an instance method on the **Dog** class.

```
class Dog:  
    def __init__(self, name):  
        self.name = name  
  
    def bark(self):  
        print("Woof!")  
  
    def get_adopted(self, owner_name):  
        self.owner = owner_name
```

Here, we use the **self** keyword inside of the **get_adopted()** instance method to refer to whichever dog this method is being called on. We set that dog's owner equal to **owner_name** just as we would assign any other variable!

Think about it: if **self** **refers to the object on which the method is being called**, and if that object is an instance of the **Dog** class, then we can call any of our other instance methods on **self**.

Here's how we'd use the **get_adopted()** method:

```
fido = Dog("Fido")  
fido.get_adopted("Sophie")  
fido.owner  
# "Sophie"
```

Optional `__init__` Arguments

There are a number of things that are important to know about a dog that you're planning to adopt. Age, weight, breed, and many more attributes factor into whether they're a good fit for you and your home. Other things, like favorite toys, are not quite so important.

To accommodate these less important details, Python lets us set default values for arguments in any method or function. Let's see how we would prepare an optional `favorite_toy` argument:

```
class Dog:
    def __init__(self, name, favorite_toy="Any"):
        self.name = name
        self.favorite_toy = favorite_toy
```

```
fido = Dog("Fido")
fido.favorite_toy
# Any
```

```
snoopy = Dog("Snoopy", "Tennis Ball")
snoopy.favorite_toy
# Tennis Ball
```

Do note, however, that arguments without default values are *not* optional:

```
old_yeller = Dog()
# TypeError: __init__() missing 1 required positional argument: 'name'
```

► *Can functions have optional arguments too?*

Instructions

Fork and clone the lab and run the tests with `pytest -x`.

1. `Dog.__init__` with Name and Breed defaulting to "Mutt"

Define a `Dog` class in `lib/dog.py`. In the class, define an `__init__` method that accepts an argument for the dog's name. That argument should be stored within a `self.name` attribute.

Additionally, `Dog.__init__` should accept a second *optional* argument for the dog's breed stored in an attribute `self.breed`. When no breed is provided, it should default to "Mutt".

2. `Person.__init__` with a Name

Define a `Person` class in `lib/person.py`. In the class, define an `__init__` method that accepts an argument for the person's name. That argument should be stored within a `self.name` attribute.

Conclusion

In Python, when we use `self` keyword in an **instance method**, `self` refers to whatever instance that method was called on. It's like a special variable that changes meaning depending on the context. Using `self` in conjunction with `__init__` allows us to create objects and set their most important attributes in one line of code. This also ensures that any objects of one class always contain the data that they need to be useful later on.

This concept of our objects being self-aware is key to our ability to write object-oriented code. It may take some time to familiarize yourself with this concept, and that's ok! Just make sure to test out your code, and find ways to determine what `self` is (like using `ipdb.set_trace()`) if you're ever not sure.

Resources

- [init in Python](https://www.geeksforgeeks.org/_init_-in-python/) ↗ [\(https://www.geeksforgeeks.org/_init_-in-python/\)](https://www.geeksforgeeks.org/_init_-in-python/)
- [Python self](https://www.w3schools.com/python/gloss_python_self.asp) ↗ [\(https://www.w3schools.com/python/gloss_python_self.asp\)](https://www.w3schools.com/python/gloss_python_self.asp)
- [What do init and self do in Python?](https://stackoverflow.com/questions/625083/what-do-init-and-self-do-in-python) ↗ [\(https://stackoverflow.com/questions/625083/what-do-init-and-self-do-in-python\)](https://stackoverflow.com/questions/625083/what-do-init-and-self-do-in-python)

This tool needs to be loaded in a new browser window

Load Object Initialization (CodeGrade) in a new window